DarnerView: Journal of Artificial Intelligence

Year 2026 • Manuscript submitted for review
e-ISSN: assigned by the journal
https://www.darnerview.com

DOI: to be assigned by the journal

RESEARCH ARTICLE

Adaptive Post-Timeout Verification under Stale Observations: Cost Gains and Unconfirmed Success in Simulation

Sepide Ebrahimian Maqsoud[1], **Moslem Mohseni Khah**[2*], **Navid Sadeghi**[3]

[1] Department of Electrical Engineering, Hamedan University of Technology, Hamedan, Iran
[2] Department of Computer Engineering, Islamic Azad University, Hamedan Branch, Hamedan, Iran
[3] Department of Electrical Engineering – Control, Shahid Beheshti University, Tehran, Iran

**Corresponding author: Moslem Mohseni Khah • Email: moslem.mohsenikhah@iau.ir*

Received: 26.10.2026 • Accepted: — • Published: —

ABSTRACT

A timed-out write can be lost, already committed or still in flight, while its status endpoint can lag behind the effect. This simulation study evaluates a particle-belief controller that selects verification time and information source by one-step expected value of information. Fixed rules and controller ablations face identical synthetic episodes with explicit loss, latency, visibility and documentation errors. Across 4,000 timed-out writes, mean recovery cost was 5.10 versus 12.28 for the development-tuned fixed wait, a 58.5% reduction under the baseline objective. The controller nevertheless made permanently false success claims in 1.95% of episodes and declared success before an eventual commit in 29.35%. The baseline objective does not charge the latter, and descriptive repricing reverses the policy ranking. Fresh-stream sensitivities add retry loss and heavier commit-latency tails without retuning the controller. They expose missing effects, escalation and prior dependence rather than an exactly-once guarantee. A separate language-model harness was implemented and tested with labelled mock completions; real-model validation was unavailable. Adaptive observation can lower assumed recovery loss, but safe completion reporting requires a separate contract or evidence condition. The findings concern this simulator and cost model, not production agent reliability.

Keywords: tool timeouts; adaptive verification; idempotency; value of information; completion reporting; simulation study

© 2026 The Author(s). This article is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0): https://creativecommons.org/licenses/by-nc/4.0/

1. Introduction

An agent sends a write request and receives a timeout. The operation might have failed before dispatch, committed without an acknowledgement, or remained in flight. A second attempt can repair the first case and duplicate the others. Reading status helps only when the read can already see the effect. This is a distributed-systems problem inherited by tool-using agents, rather than a new failure caused by language generation. Idempotent processing, compensation and explicit consistency contracts are established responses (Garcia-Molina & Salem, 1987; Helland, 2007; Vogels, 2009; Stripe, n.d.).

The unresolved decision in this study is narrower: how much observation should a client buy after the timeout, and when? Immediate verification is cheap but may be stale; waiting can reveal the effect but costs time; an independent ledger can remove visibility lag while still missing an in-flight write. A controller can choose among these actions using a belief about latency and loss. Its expected-cost advantage may nevertheless depend on treating an operation that will eventually commit as already successful. We therefore distinguish eventual outcomes from what has been established when success is declared.

The contribution is a controlled measurement of this observation decision and its objective dependence. Verify-before-retry is prior work (Mansoor et al., 2026), and the impossibility of verification-only exactly-once completion under unbounded late commits is established by Li (2026). We do not claim either principle as new. We compare a simple belief controller with fixed rules, isolate its components and expose conditions in which a low reported recovery cost is a misleading reliability summary.

1) RQ1: Does adaptive verification reduce the stated recovery cost relative to fixed policies on paired timeout episodes?
2) RQ2: Which information components contribute, and how do permanently false and premature success claims limit that gain?
3) RQ3: What changes under learned or wrong priors, lost retries, heavier latency tails and alternative confirmation costs?

2. Literature Review

2.1. Distributed effects and uncertainty

Sagas organize long work as local transactions with compensating actions (Garcia-Molina & Salem, 1987); compensation is different from preventing an excess effect. Helland (2007) describes durable partner state for coping with repeated messages and makes uncertainty part of application semantics. Vogels (2009) distinguishes consistency guarantees that determine what a client can infer from a read. Stripe (n.d.) documents an operational idempotency-key interface. Our simulator abstracts those concerns into loss, delayed commit, status lag and imperfect documentation. It does not reproduce a transaction manager or a payment API.

2.2. Verification wrappers and late commits

Mansoor et al. (2026) combine postcondition checks, verify-before-retry and stable keys around a language-model agent in a simulated tool environment. Their verifier exposes inconclusive states, and the paper already proposes confidence-based choices among retry, further verification and intervention as future work. Our belief controller instantiates a restricted version of that proposed direction; the conceptual idea of adaptive verification is therefore not novel.

Li (2026) separately varies model, harness and contract in the Limbo sandbox, distinguishing current-state from eventual-outcome oracles. Its late-commit result shows that a read which is accurate now may still be insufficient for safe retry. Its waiting and contract interventions are closer comparators than generic tool-use benchmarks. Our experiments differ by explicitly pricing observation time/source and measuring a parametric controller. We neither reproduce Limbo nor infer model performance from its published results.

2.3. Decision-theoretic observation

Expected information value compares decision loss before and after an observation (Raiffa & Schlaifer, 2000). Particle representations support non-Gaussian beliefs (Gordon et al., 1993), but our controller uses rejection and conditioning on a static latent world, without the resampling dynamics of a general particle filter. Its one-step calculation is an approximation to sequential decision control. The study asks whether that approximation buys lower cost under stated assumptions, not whether it is an optimal controller.

3. Materials and Methods

3.1. Paired design and seed separation

The main experiment contains 4,000 independent timeout episodes, with seeds starting at 0. All policies share the sampled service and latent outcome. The fixed-wait duration is tuned on 600 development episodes beginning at seed 900000, among waits of 0, 1, 2, 4, 8, 12, 16 and 24 seconds; it is then frozen. Learned-prior, median and loss-cost sensitivities reuse paired seed streams. Added retry-loss and tail-shape experiments use fresh streams beginning at 50000 and 60000, with 2,000 episodes per condition. These additions are exploratory revisions, fixed before their outcomes were inspected; the study was not preregistered.

3.2. Executable service model

A write is dispatched at time 0 and times out at 2 seconds. Commit latency is lognormal; status visibility adds either no lag or an exponential lag with mean 1 or 3 seconds. Three equally selected service patterns use request-loss probabilities 0.03, 0.03 and 0.25; acknowledgement-loss probabilities 0.15, 0.02 and 0.05; latency medians 0.3, 1.5 and 0.5 seconds; and log-scale standard deviations 0.5, 1.0 and 0.6. Rejection sampling conditions worlds on loss, acknowledgement loss or a response arriving after the timeout. Tests retain the acknowledgement evidence and check this condition directly.

Half the services document idempotency; the promise is honored in 90% of those cases. An in-flight bound of 6 or 12 seconds is documented for 30% of services and enforced for 90% of that subset. Policies see documentation and lag characteristics, not whether a promise is true. A status read never reports a nonexistent effect but may lag. Independent evidence sees every commit so far, including ones not visible through status, and cannot observe a future commit. These ideal read properties intentionally simplify verification.

3.3. Decisions, ledger and objective

The client may wait, read status, read independent evidence, declare success, retry once or escalate. Status costs 2 units and 0.2 seconds; evidence costs 6 and 1 second; each second costs 1. A retry costs 1. A duplicate or permanently false success costs 100, and escalation costs 25 as a terminal handoff, without simulating the human resolution. The baseline retry is assumed to land. The evaluator counts eventual effects, including the original if it commits after the decision. It separately records a success declaration before that commit as premature. Such premature declarations have no baseline penalty, an explicit objective limitation.

The added retry-loss experiment gives the one retry a shared, independent latent loss event with probability 0, 0.05, 0.15 or 0.30. If both requests are lost, a retry leaves no effect and incurs a missing-effect charge of 100. A lost retry cannot create a duplicate. The adaptive controller knows the probability, not the realized outcome; fixed rules remain unchanged. This is a single-attempt sensitivity, not a model of repeated transport recovery.

3.4. Policies and belief controller

- Immediate retry; one status probe then retry if absent; the development-tuned wait then probe; an additional short fixed wait; and a contract guard form the five fixed comparators.

- The contract guard retries documented keyed writes, otherwise waits out a documented bound before reading evidence, or waits briefly and probes when no bound is documented.
- The adaptive controller keeps 1500 timeout-conditioned particles over loss, commit time and status lag. A present read terminates successfully; an absent read removes inconsistent particles.
- Ablations remove the status-lag model, independent evidence or contract information. The outcome oracle sees eventual truth and avoids unnecessary retries; it is an unattainable reference for the baseline objective.

$$L_S = C_F\, p; \quad L_R = C_R + (1-r)(1-\pi)C_D(1-p) + r\, C_M\, p \tag{1}$$

Here p is loss probability; C_F, C_R, C_D and C_M price false success, retry, duplication and a missing retry. The symbol r is retry-loss probability and π is assumed key reliability, 0.9 when documented and zero otherwise. Escalation has fixed loss. The controller evaluates each wait of 0, 1, 3 or 8 seconds followed by status or evidence, computes expected posterior terminal loss and subtracts observation and delay costs. It executes the best positive-value observation and repeats, for at most eight steps. Fewer than ten surviving particles cause escalation. The one-step criterion omits the value of later observations and does not penalize an eventual-but-unconfirmed success.

3.5. Outcomes, inference and sensitivities

Mean recovery cost is primary. The primary family contains all eight paired cost contrasts of adaptive against the five fixed policies and three ablations; the oracle is excluded. Percentile intervals use 2000 paired episode resamples, and sign-flip tests use 2000 permutations with Holm correction over the full family (Efron, 1979; Holm, 1979). Intervals are nominal rather than simultaneous. Risk intervals use the score method (Wilson, 1927). Independent generated episodes are the inference unit; results are not population estimates for deployed services.

Secondary outcomes include duplicates, permanently false success, premature success, escalation, missing effects and observation/time costs. Priors are known, fitted from 20, 60 or 300 unconditioned historical writes, or wrong by latency-median factors 0.25, 0.5, 2 and 4. Duplicate and false-success charges vary together from 10 to 1000. The fresh tail sensitivity multiplies actual lognormal sigma by 1, 1.5 or 2 while the controller retains baseline sigma. It therefore changes both distribution shape and mean, not only rare-tail frequency.

A separate descriptive rescoring adds 0, 25 or 100 units per premature declaration to the recorded trajectories. No policy is retuned for this new loss; it tests whether the baseline ranking survives a different reporting requirement. These risk and sensitivity analyses are exploratory and use pointwise intervals. They are not additional independent confirmations of the primary hypothesis.

3.6. Separate model protocol and ethics

The model harness uses 200 fresh episodes beginning at 70000. It reveals only contract documentation, the timeout, current time and requested observations. A model selects bounded JSON actions; the simulator alone scores effects and costs, alongside the same adaptive and fixed baselines. Raw completions and prompt hashes are cached, temperature is zero, and paid requests require explicit prices under a persistent budget cap. No credentials or ready local runtime were

found; official runtime acquisition timed out. A labelled mock checks parsing and scoring only. All tasks and data are synthetic; there are no human participants or personal data.

4. Results

4.1. Primary cost comparison

Table 1. *Baseline recovery loss, eventual errors and elapsed time*

Policy	Cost [95% interval]	Duplicate %	False success %	Seconds
Immediate retry	35.15 [33.75, 36.62]	34.2	0.0	0.00
Probe then retry	22.54 [21.36, 23.82]	19.6	0.0	0.20
Short fixed wait	15.55 [14.64, 16.56]	10.8	0.0	2.20
Tuned fixed wait	12.28 [11.91, 12.69]	1.7	0.0	8.20
Contract guard	14.09 [13.18, 15.02]	9.6	0.0	2.06
No staleness model	7.90 [7.17, 8.67]	4.2	1.9	0.34
No evidence	5.38 [4.87, 5.95]	0.9	1.9	1.05
No contract	5.56 [5.14, 6.01]	0.1	2.0	0.96
Adaptive	5.10 [4.65, 5.62]	0.6	1.9	0.62
Outcome oracle	0.37 [0.36, 0.39]	0.0	0.0	0.00

Note. False success means the operation never commits. Premature declarations are reported separately and are not charged in this objective.

For RQ1, adaptive cost was 5.10 [4.65, 5.62] against 12.28 [11.91, 12.69] for the 8-second tuned wait (Table 1), a 58.5% reduction. The paired difference was −7.18 [−7.82, −6.53], with standardized paired effect −0.349 and Holm p = 0.0040. This is a cost result under the baseline objective; it does not establish reliable completion reporting.

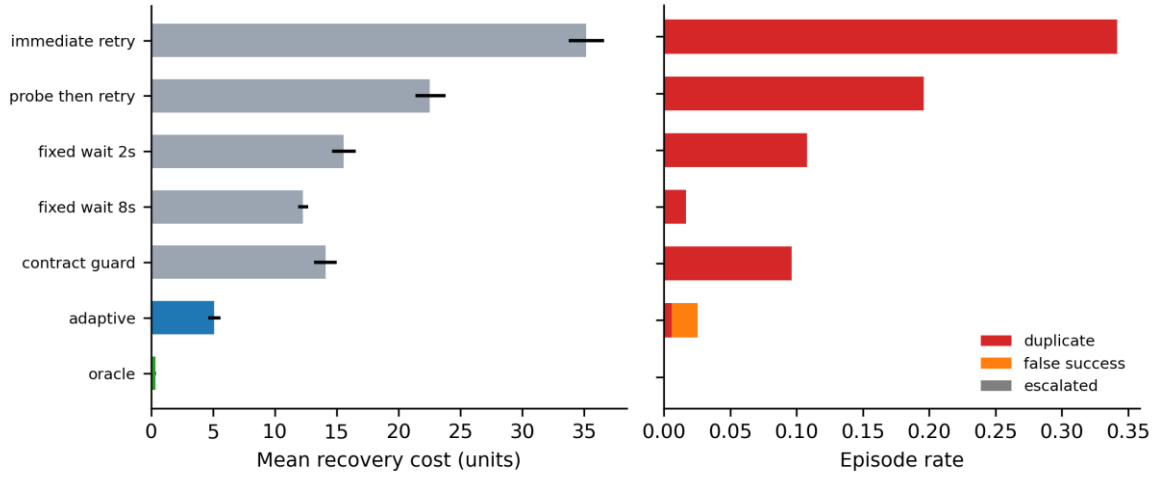


Figure 1. *Mean baseline recovery cost with intervals and eventual error rates. Premature confirmation is excluded from the plotted cost and must be read alongside the separate risk analysis.*

4.2. Information components and reporting risk

For RQ2, removing the staleness model raised cost to 7.90, versus 5.10 for the full controller. Omitting evidence or contract information produced costs 5.38 and 5.56. All eight paired estimates and corrected tests are supplied; component contributions are conditional on this controller and do not establish additive causal shares.

The controller made permanently false success claims in 1.95% [1.6, 2.4] of episodes. It also declared success before a later commit in 29.35% [28.0, 30.8]. The latter is not an eventual missing operation, but it is unverified completion at decision time. The baseline gain depends on allowing that distinction.

Table 2. *Descriptive repricing of existing trajectories for premature success*

Added charge	Adaptive cost [interval]	Tuned-wait cost
0	5.10 [4.62, 5.59]	12.28
25	12.44 [11.91, 12.98]	12.28
100	34.45 [33.08, 35.83]	12.28

Note. Policies were optimized or tuned for the baseline loss and are not retuned here. The outcome oracle is also baseline-objective-specific.

The policy ranking reverses under a charge for premature reporting (Table 2). This is a construct-validity finding, not evidence that the original controller remains optimal under the revised loss. A policy that requires positive evidence before declaring completion would solve a different constrained decision problem.

4.3. Priors, retry loss and heavier tails

For RQ3, the controller cost 6.85 with priors fitted from 20 historical writes. Across the latency-median sensitivity its largest mean cost was 7.35. These are paired sensitivity streams rather than independent replications of the main estimate. The supplement reports their corresponding fixed rules and the loss-charge sweep; the plots show dependence on both data volume and latency calibration.

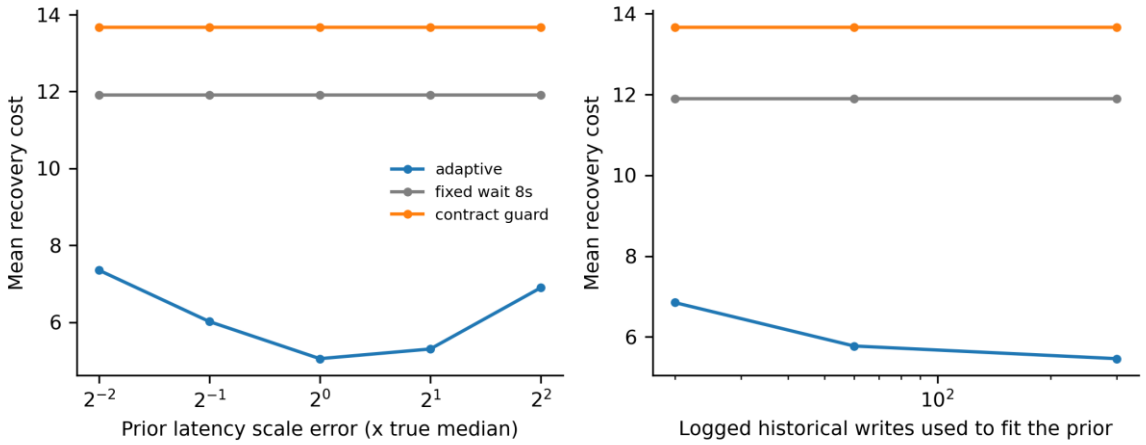


Figure 2. *Mean recovery cost under latency-median misspecification and priors estimated from historical writes. These plots retain the baseline objective and do not price premature confirmation.*

Table 3. *Fresh-stream lost-retry sensitivity for the adaptive controller*

Retry loss	Cost [interval]	Missing retry %	Escalation %	False success %
0.00	5.22 [4.52, 5.90]	0.0	0.0	2.1
0.05	7.07 [6.11, 8.03]	1.9	0.0	2.1
0.15	9.53 [8.41, 10.74]	4.5	0.0	2.1
0.30	13.32 [12.52, 14.15]	0.0	37.4	2.1

Note. The retry loss probability is known; its realization is hidden. Missing retry means both the original and its retry were lost. False success refers to a terminal success decision.

Lost retries increase recovery loss and change the choice between retry and handoff (Table 3). A lower missing-retry rate at the largest loss probability can result from escalation, so it must not be read as improved autonomous completion. The policy uses only one retry and does not estimate unknown transport reliability.

When actual sigma doubled while the prior stayed fixed, adaptive cost changed from 5.02 [4.38, 5.69] to 8.27 [7.23, 9.30]. Premature-success rates changed from 30.7% to 31.6%. The test therefore supports sensitivity, not blanket robustness to heavy tails. Its paired fixed-policy contrasts and risk intervals are in the supplement.

4.4. Model execution status

No real language-model decisions were obtained. The mock run exercises the public observation boundary, action validation, cached transcripts and exact ledger scoring. Mock outcomes are excluded from every scientific estimate above. Consequently the manuscript names no model and reports no model confidence interval.

5. Discussion

Adaptive observation lowers expected loss when different services require different evidence and waiting times. Its advantage comes primarily from not interpreting stale absence as proof that nothing happened. However, the controller also exploits a permissive terminal objective: a likely future commit can be called successful before it occurs. Separating this behavior from permanent false success changes the interpretation of the cost improvement. The result is a trade-off between the modeled loss and completion assurance, rather than evidence that adaptation makes retries safe.

The impossibility boundary is central. On a non-idempotent write with no trusted in-flight bound, an absent read is compatible with both a permanently lost request and one that will commit later. Any finite policy that eventually reissues the lost request also reissues in a sufficiently delayed world with the same observations, creating a duplicate when both land (Li, 2026). Independent evidence can remove status lag but cannot observe the future. Our controller does not evade this argument; it accepts expected risk, sometimes declares an unconfirmed success, or can hand off.

Stable idempotency and durable operation identity address a different layer of the problem (Helland, 2007; Stripe, n.d.). A learned prior can improve a decision under uncertainty, but it cannot turn an advisory or false contract into a guarantee. The wrong-documentation breakdown in the supplement measures that exposure. The model-driven harness would test whether textual contract use and observation choices resemble this idealized controller; without real-model calls, no such transfer has been demonstrated.

6. Conclusion

RQ1 has a conditional positive answer: baseline adaptive cost was 5.10 against 12.28 for the tuned fixed wait. RQ2 identifies the staleness model as the largest measured component while exposing 1.95% permanently false and 29.35% premature success declarations. RQ3 finds sensitivity to lost retries, actual tail shape and the cost of unconfirmed reporting. The ranking reversal under repricing restricts the main claim to the stated objective. Exactly-once completion remains a contract and evidence question.

6.1. Recommendations

Evaluate observation policies with both an effect ledger and a decision-time reporting metric. Choose the loss function before tuning, state whether future commits count as success, and keep latency calibration separate from final evaluation. Where verified completion is required, add a positive-evidence condition or a trusted operation contract and evaluate the resulting controller anew. Future model studies should retain raw action traces and report whether the simulation ranking survives actual language-model decisions.

6.2. Limitations

Only three parametric service patterns and one write per episode are modeled. Reads have perfect specificity, independent evidence is authoritative and correlation between faults is omitted. Contract and latency parameters are largely known, historical logs are uncensored and retry loss is supplied rather than learned. These favor a calibrated controller. The primary cost ignores premature reporting, does not simulate human resolution after escalation and uses abstract charges rather than measured money or wall-clock latency. Finite particles, one-step lookahead and fixed action limits restrict optimization quality.

The prior paper draft had already explored these mechanisms; this revision is not a preregistered confirmation. New retry/tail streams are separated but still generated by the same model family. Repricing is a diagnostic of frozen trajectories, not a fair retuning comparison under a new objective. Doubling lognormal sigma changes the mean as well as tails. No real-model cohort or production workload was obtained, so prompt, model, provider and deployment generalisation are unmeasured. The intervals quantify simulated episode variability and cannot establish safety guarantees.

Declarations

Ethics Committee Approval: Not applicable: synthetic tasks and simulated data; no human participants or personal data were involved.

Author Contributions: The authors take collective responsibility for this manuscript.

Conflict of Interest: The authors declare no conflict of interest.

Funding: This research received no external funding.

Data and Code Availability: Code, data and generated results: https://github.com/moslem-mohseni/adaptive-post-timeout-verification. Supplementary material: supplementary_p3.zip; also available from the corresponding author.

References

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, *7*(1), 1–26. https://doi.org/10.1214/aos/1176344552

Garcia-Molina, H., & Salem, K. (1987). Sagas. *ACM SIGMOD Record*, *16*(3), 249–259. https://doi.org/10.1145/38714.38742

Gordon, N. J., Salmond, D. J., & Smith, A. F. M. (1993). Novel approach to nonlinear/non-Gaussian Bayesian state estimation. *IEE Proceedings F (Radar and Signal Processing)*, *140*(2), 107–113. https://doi.org/10.1049/ip-f-2.1993.0015

Helland, P. (2007). Life beyond distributed transactions: An apostate's opinion. In *Proceedings of the 3rd Biennial Conference on Innovative Data Systems Research* (pp. 132–141). https://www.cidrdb.org/cidr2007/papers/cidr07p15.pdf

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, *6*(2), 65–70. https://www.jstor.org/stable/4615733

Li, J. (2026). Where does exactly-once live? Model, harness, and tool-contract effects on duplicate side effects in LLM agents. *arXiv*. https://arxiv.org/abs/2609.29095v1

Mansoor, I. K., Phadke, A., & Rana, P. (2026). Verified tool calls improve LLM agent reliability under non-atomic failures. *arXiv*. https://arxiv.org/abs/2608.02645v1

Raiffa, H., & Schlaifer, R. (2000). *Applied statistical decision theory*. Wiley. https://www.wiley.com/en-us/applied-statistical-decision-theory-p-9780471383499

Stripe. (n.d.). *Idempotent requests*. Stripe API Reference. https://docs.stripe.com/api/idempotent_requests

Vogels, W. (2009). Eventually consistent. *Communications of the ACM*, *52*(1), 40–44. https://doi.org/10.1145/1435417.1435432

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, *22*(158), 209–212. https://doi.org/10.1080/01621459.1927.10502953